

Writing your own library

Morten Hjorth-Jensen¹

Department of Physics, University of Oslo, Norway¹

June 12, 2025

© 1999-2025, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Running on the IBM machines

For the projects and exercises, using for example Qiskit, we can run (as of now) on IBM's 127 qubit machine (Osaka). The Bell-state simulations below here perform this. To get started with the IBM machines, you need an account. Here's the recipe you need to follow:

1. Go to quantum.ibm.com and create an account and follow the instructions.
2. After you have set up your account, go to the account icon and find the token. You need this to identify your self when running jobs. See the code example below.
3. Clicking on the **jobs** link you can see which machines are available and where your job is, whether it is a queue or whether it is pending.

The example at the end of this week's slides shows you how can run on IBM-Osaka, the 127 qubit machine which is presently available. Make sure you copy in your own token, as that is your unique identifier.

Explicit instructions

```
# Install qiskit and various packages  
pip install -q qiskit  
pip install -q qiskit-aer  
pip install -q qiskit-ibmq-provider
```

Setting up your IBM run

```
# Provide your IBM Quantum Experience API token here  
# Paste in this from your IBM account  
api_token = 'your_api_token'  
# Load your IBM Quantum account  
IBMQ.save_account(api_token, overwrite=True) # This stores your credentials  
IBMQ.load_account()  
# Get the least busy backend  
provider = IBMQ.get_provider(hub='ibm-q', group='open', project='main')  
# Run on the 127-qubit machine IBM-Osaka  
backend = provider.get_backend('ibm_osaka')  
# Create a quantum circuit with two qubits  
bell_circuit = QuantumCircuit(2, 2)
```

Quantum gates, circuits and simple algorithms

Quantum gates are physical actions that are applied to the physical system representing the qubits. Mathematically, they are complex-valued, unitary matrices which act on the complex-valued normalized vectors that represent qubits. As the quantum analog of classical logic gates (such as AND and OR), there is a corresponding quantum gate for every classical gate; however, there are quantum gates that have no classical counter-part. They act on a set of qubits and, changing their state. That is, if U is a quantum gate and $|q\rangle$ is a qubit, then acting the gate U on the qubit $|q\rangle$ transforms the qubit as follows:

$$|q\rangle \xrightarrow{U} U|q\rangle. \quad (1)$$

Quantum circuits

Quantum circuits are diagrammatic representations of quantum algorithms. The horizontal dimension corresponds to time; moving left to right corresponds to forward motion in time. They consist of a set of qubits $|q_n\rangle$ which are stacked vertically on the left-hand side of the diagram. Lines, called quantum wires, extend horizontally to the right from each qubit, representing its state moving forward in time. Additionally, they contain a set of quantum gates that are applied to the quantum wires. Gates are applied chronologically, left to right.

Single-Qubit Gates

A single-qubit gate is a physical action that is applied to one qubit. It can be represented by a matrix U from the group $SU(2)$. Any single-qubit gate can be parameterized by three angles: θ , ϕ , and λ as follows

$$U(\theta, \phi, \lambda) = \begin{bmatrix} \cos \frac{\theta}{2} & -e^{i\lambda} \sin \frac{\theta}{2} \\ e^{i\phi} \sin \frac{\theta}{2} & e^{i(\phi+\lambda)} \cos \frac{\theta}{2} \end{bmatrix}.$$

Widely used gates

There are several widely used quantum gates. Perhaps the most famous are the Pauli gates correspond to the Pauli matrices

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Algebra basis

These gates form a basis for the algebra $\mathfrak{su}(2)$. Exponentiating them will thus give us a basis for $SU(2)$, the group within which all single-qubit gates live.

Exponentiated Pauli gates

These exponentiated Pauli gates are called rotation gates $R_\sigma(\theta)$ because they rotate the quantum state around the axis $\sigma = X, Y, Z$ of the Bloch sphere by an angle θ . They are defined as

$$R_X(\theta) = e^{-i\frac{\theta}{2}X} = \begin{bmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix},$$

$$R_Y(\theta) = e^{-i\frac{\theta}{2}Y} = \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix},$$

$$R_Z(\theta) = e^{-i\frac{\theta}{2}Z} = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}.$$

Basis for SU(2)

Because they form a basis for SU(2), any single-qubit gate can be decomposed into three rotation gates. Indeed

$$R_z(\phi)R_y(\theta)R_z(\lambda) = \begin{bmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{bmatrix} \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix} \begin{bmatrix} e^{-i\lambda/2} & 0 \\ 0 & e^{i\lambda/2} \end{bmatrix}$$

which we can rewrite as

$$e^{-i(\phi+\lambda)/2} \begin{bmatrix} \cos \frac{\theta}{2} & -e^{i\lambda} \sin \frac{\theta}{2} \\ e^{i\phi} \sin \frac{\theta}{2} & e^{i(\phi+\lambda)} \cos \frac{\theta}{2} \end{bmatrix},$$

which is, up to a global phase, equal to the expression for an arbitrary single-qubit gate.

Two-Qubit Gates

A two-qubit gate is a physical action that is applied to two qubits. It can be represented by a matrix U from the group $SU(4)$. One important type of two-qubit gates are controlled gates, which work as follows: Suppose U is a single-qubit gate. A controlled- U gate (CU) acts on two qubits: a control qubit $|x\rangle$ and a target qubit $|y\rangle$. The controlled- U gate applies the identity I or the single-qubit gate U to the target qubit if the control gate is in the zero state $|0\rangle$ or the one state $|1\rangle$, respectively.

Control qubit

The control qubit is not acted upon. This can be represented as follows if

$$CU|xy\rangle = |xy\rangle \text{ if } |x\rangle = |0\rangle.$$

In matrix form

It is easier to see in a matrix form. It can be written in matrix form by writing it as a superposition of the two possible cases, each written as a simple tensor product

$$CU = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{bmatrix}.$$

CNOT gate

One of the most fundamental controlled gates is the CNOT gate. It is defined as the controlled- X gate CX . It can be written in matrix form as follows:

$$\text{CNOT} = CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

CX gate

It changes, when operating on a two-qubit state where the first qubit is the control qubit and the second qubit is the target qubit, the states (check this)

$$\text{CX}|00\rangle = |00\rangle,$$

$$\text{CX}|10\rangle = |11\rangle,$$

$$\text{CX}|01\rangle = |01\rangle,$$

$$\text{CX}|11\rangle = |10\rangle,$$

which you can easily see by simply multiplying the above matrix with any of the above states.

Swap gate

A widely used two-qubit gate that goes beyond the simple controlled function is the SWAP gate. It swaps the states of the two qubits it acts upon

$$\text{SWAP}|xy\rangle = |yx\rangle.$$

and has the following matrix form

$$\text{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Introduction to Qiskit

This part is best seen using the jupyter-notebook.

For the installation

```
pip install qiskit  
pip install qiskit-ibm-runtime
```

Documentation can be found at

<https://docs.quantum.ibm.com/api/qiskit>

Simple code

```
#!/usr/bin/env python
# coding: utf-8
import numpy as np
import qiskit as qk
from scipy.optimize import minimize

# # Initialize registers and circuit

n_qubits = 1 #Number of qubits
n_cbits = 1 #Number of classical bits (the number of qubits you want t
qreg = qk.QuantumRegister(n_qubits) #Create a quantum register
creg = qk.ClassicalRegister(n_cbits) #Create a classical register
circuit = qk.QuantumCircuit(qreg,creg) #Create your quantum circuit

circuit.draw() #Draw circuit. It is empty
```

Thereafter we perform operations on qubit

```
circuit.x(qreg[0]) #Applies a Pauli X gate to the first qubit in the q
circuit.draw()
```

and select a qubit to measure and encode the results to a classical bit

```
#Measure the first qubit in the quantum register
#and encode the results to the first qubit in the classical register
```

Exercises on Bell states

```
import qiskit as qk
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
import matplotlib.pyplot as plt
```

Setting up a circuit

```
qreg_q = QuantumRegister(2) # 2 qubits
creg_c = ClassicalRegister(2) # 2 classical bits

qc = QuantumCircuit(qreg_q, creg_c) # alternatively, circuit = QuantumCircuit(qreg_q, creg_c)

qc.h(qreg_q[0]) # apply the hadamard gate to the first qubit (to the rest of the qubits)

# apply the CNOT gate with the first qubit being the control qubit and the second qubit being the target qubit
qc.cx(qreg_q[0], qreg_q[1])

# measure the qubits, specify which qubit you want to measure
qc.measure(qreg_q[0], creg_c[0])
qc.measure(qreg_q[1], creg_c[1])
```

Next we will use an ideal(noiseless) simulator (AerSimulator) to run our circuit. We will use the qasm_simulator backend. This simulator returns a result object which contains the counts, or

Testing the Bell states

```
from qiskit import QuantumCircuit, transpile, assemble, IBMQ
from qiskit.visualization import circuit_drawer
import qiskit_aer
import matplotlib.pyplot as plt

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
qubit as the target
bell_circuit.cx(0, 1)

# Add measurements to the circuit
bell_circuit.measure([0, 1], [0, 1])

# Visualize the circuit
print("Quantum Circuit:")
print(bell_circuit)
```

Running the calculations

```
# Number of shots
num_shots = 10000

# Simulate the circuit using the Aer simulator
simulator = qiskit_aer.Aer.get_backend('qasm_simulator')

# Transpile the circuit for the simulator
transpiled_circuit = transpile(bell_circuit, simulator)

# Execute the transpiled circuit on the simulator with the specified n
result = simulator.run(transpiled_circuit, shots=num_shots).result()

# Get measurement counts
counts = result.get_counts(bell_circuit)

# Get and print the measurement results
counts = result.get_counts(bell_circuit)
print("\nMeasurement Results:")
print(counts)
```

Then plotting the histogram over counts

```
# Plot the histogram using Matplotlib  
plt.bar(counts.keys(), counts.values())  
plt.title('Bell State Measurement Results')  
plt.ylabel('Counts')  
plt.show()
```

Getting serious, Quantum Simulator with Hardware noise model

```
from qiskit import QuantumCircuit, transpile, assemble, Aer, IBMQ
from qiskit.visualization import plot_histogram
from qiskit.providers.aer.noise import NoiseModel

# you will not be able to run unless you provide your token here
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True) # This stores your credentials
IBMQ.load_account()
# Get the least busy backend
provider = IBMQ.get_provider(hub='ibm-q', group='open', project='main')
#backend = provider.get_backend('your_preferred_backend')
backend = provider.get_backend('ibm_osaka')

# Get the noise model for the selected backend
noise_model = NoiseModel.from_backend(backend)

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second
bell_circuit.cx(0, 1)
```


Then run and visualize it

```
# Number of shots
```

```
num_shots = 10000
```

```
# Simulate the circuit using the Aer simulator with the noise model
```

```
simulator = Aer.get_backend('qasm_simulator')
```

```
noisy_transpiled_circuit = transpile(bell_circuit, simulator, optimization_level=3)
```

```
noisy_circuit = assemble(noisy_transpiled_circuit, shots=num_shots, noise_model=noise_model)
```

```
# Execute the noisy circuit on the simulator
```

```
result = simulator.run(noisy_circuit).result()
```

```
# Get and print the measurement results
```

```
counts = result.get_counts(bell_circuit)
```

```
print("\nMeasurement Results:")
```

```
print(counts)
```

```
# Plot the histogram using Matplotlib
```

```
plot_histogram(counts, title='Bell State Measurement Results')
```

And without the noise model

```
# Provide your IBM Quantum Experience API token here
api_token = 'your_api_token'
# Load your IBM Quantum account
IBMQ.save_account(api_token, overwrite=True) # This stores your credentials
IBMQ.load_account()

# Get the least busy backend
#provider = IBMQ.get_provider(hub='your_hub', group='your_group', project='your_project')
provider = IBMQ.get_provider(hub='ibm-q', group='open', project='main')
#backend = provider.get_backend('your-preferred-backend')
backend = provider.get_backend('ibm_osaka')

# Create a quantum circuit with two qubits
bell_circuit = QuantumCircuit(2, 2)

# Apply Hadamard gate to the first qubit
bell_circuit.h(0)

# Apply a CNOT gate with the first qubit as the control and the second qubit as the target
bell_circuit.cx(0, 1)

# Add measurements to the circuit
bell_circuit.measure([0, 1], [0, 1])

# Transpile the circuit for the chosen backend
transpiled_circuit = transpile(bell_circuit, backend)

# Execute the transpiled circuit on the IBM Quantum computer
```

Setting up an object-oriented environment

Here we set up the basic elements for implementing a simple quantum circuit simulator in Python using object-oriented design. We simulate quantum state evolution using only NumPy matrix operations, without relying on specialized quantum libraries like Qiskit or PennyLane. A quantum state of n qubits is represented as a length 2^n complex state vector, and each quantum gate corresponds to a unitary matrix that transforms this state vector. We support all standard one-qubit gates (including identity I , Pauli X , Y , Z , Hadamard H , phase gates S , T , and arbitrary rotations R_x , R_y , R_z), as well as common two-qubit gates (controlled-NOT, controlled-Z, SWAP). The simulator allows composing circuits from these gates and visualizing results (either on the Bloch sphere for single qubits or as state-vector probabilities for multi-qubit states). All code is written with standard Python, using NumPy for linear algebra and Matplotlib for visualization.

One-Qubit Gates

A single-qubit gate is represented by a 2×2 unitary matrix acting on the two-dimensional Hilbert space spanned by $|0\rangle$ and $|1\rangle$. For example, the Pauli- X gate (quantum NOT) has matrix $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, which flips the amplitudes of $|0\rangle$ and $|1\rangle$. Similarly, $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ and $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ are 180° rotations about the y - and z -axes of the Bloch sphere, with Z applying a phase flip to $|1\rangle$ (changing its sign). The Hadamard gate $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ creates superposition: $H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$ and $H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$. We also include the phase gates S and T , which are quarter-turn Z -rotations: $S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$ (90° phase shift on $|1\rangle$) and $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$ (45° phase shift). These satisfy $S^2 = Z$ and $T^2 = S$. For arbitrary rotations, we support parameterized gates $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$, which rotate a qubit by angle θ about the x , y , or z axis of the Bloch sphere. For instance, $R_x(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ i\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$ and $R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$. (The R_y matrix is similarly $\begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$). An identity gate I

Our first class

Below is the implementation of a Qubit class (for labeling qubits) and a base Gate class with subclasses for each one-qubit gate. Each gate class stores its 2×2 unitary matrix and the target qubit it acts on. We use NumPy to represent matrices and will apply them to state vectors later. (Note: We allow specifying the target as either a Qubit object or an integer index for convenience.)

```
import numpy as np
```

```
class Qubit:
```

```
    def __init__(self, index):
```

```
        self.index = index
```

```
class Gate:
```

```
    """Base class for quantum gates."""
```

```
    def __init__(self, matrix, targets):
```

```
        self.matrix = np.array(matrix, dtype=np.complex128)
```

Two-Qubit Gates

Two-qubit gates act on a 4-dimensional joint state space of two qubits (basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$). The most important two-qubit gate is the Controlled-NOT (CNOT), which flips a target qubit if and only if the control qubit is $|1\rangle$. In matrix form (with control as the first qubit and target as second), CNOT is:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix},$$

which swaps the amplitudes of $|10\rangle$ and $|11\rangle$ (flipping the target when the control is 1). Another common entangler is Controlled-Z (CZ), which applies a Z phase flip to the target qubit when the control is 1. Its matrix is diagonal $\text{diag}(1, 1, 1, -1)$ in the $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ basis, giving a -1 phase to $|11\rangle$ (and leaving the other basis states unchanged). We also implement the SWAP gate, which swaps the states of two qubits. In basis form, SWAP sends $|a, b\rangle$ to $|b, a\rangle$; its matrix is

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Code for two-qubit gates

Below is the code for two-qubit gate classes (CNOTGate, CZGate, SWAPGate and a general TwoQubitGate), followed by the core Circuit class. Each gate class defines the appropriate 4×4 matrix. The Circuit class encapsulates a list of qubits and gates, along with the current state vector of the system. By default, the circuit's qubits start in the state $|00 \dots 0\rangle$

(represented as a state vector with 1 at index 0 and 0 elsewhere). The `Circuit.add_gate()` method appends gates to the circuit, and `Circuit.run()` applies

State evolution: To apply a gate to the full state vector, we perform matrix operations on the state. For example, applying a one-qubit gate U on qubit j in an n -qubit state corresponds to the operation $I \otimes \dots \otimes U_j \otimes \dots \otimes I$ on the combined Hilbert space, effectively inserting U on the j th qubit's subspace and identity on the others. We implement this by manipulating the state vector indices. Similarly, a two-qubit gate on qubits (p, q) is applied as $I \otimes \dots \otimes U_{(p,q)} \otimes \dots \otimes I$. The code efficiently updates the state vector without constructing the full $2^n \times 2^n$ matrix explicitly: it groups amplitudes into pairs (for one-qubit gates) or quadruples (for two-qubit gates) corresponding to the basis states of the target

Two-qubit gate subclasses:

```
class CNOTGate(Gate):
```

```
    """Controlled-NOT gate: flips target if control is 1."""
```

```
    def __init__(self, control, target):
```

```
        matrix = [[1, 0, 0, 0],
```

```
                  [0, 1, 0, 0],
```

```
                  [0, 0, 0, 1],
```

```
                  [0, 0, 1, 0]]
```

```
        super().__init__(matrix, [control, target])
```

```
        self.name = "CNOT"
```

```
class CZGate(Gate):
```

```
    """Controlled-Z gate: phase flip on |11>."""
```

```
    def __init__(self, control, target):
```

```
        matrix = [[1, 0, 0, 0],
```


Example Usage and Visualization

To demonstrate the simulator, we construct two example circuits: one single-qubit circuit and one two-qubit circuit. We then run the circuits and visualize the results.

Single-Qubit Circuit Example

For the single-qubit example, we apply a Hadamard gate to an initial $|0\rangle$ state. This should create the state $\frac{|0\rangle+|1\rangle}{\sqrt{2}}$, which is often called $|+\rangle$ (a superposition along the x-axis of the Bloch sphere). After running the circuit, the state vector is approximately $[0.707 + 0j, ; 0.707 + 0j]$, meaning equal amplitude (and probability 0.5 each) for $|0\rangle$ and $|1\rangle$. We visualize this state on the Bloch sphere.

Bloch sphere representation of the single-qubit state after applying a Hadamard gate. The state vector (shown in purple) points along the positive X-axis, corresponding to the superposition $\frac{|0\rangle+|1\rangle}{\sqrt{2}}$ (often denoted $|+\rangle$). On the Bloch sphere, the north pole represents $|0\rangle$ (pointing along +Z) and the south pole $|1\rangle$ (-Z). Here the state lies on the equator at +X, as expected for $|+\rangle$. The simulator computed Bloch coordinates $(x, y, z) \approx (1, 0, 0)$ for this state, indicating an X-axis alignment (and zero Y or Z components).

Single-qubit circuit: start in $|0\rangle$, apply H gate

```
circuit1 = Circuit(1)
```

Two-Qubit Circuit Example

For the two-qubit example, we create an entangled Bell state. We start with two qubits in $|00\rangle$. Then we apply a Hadamard on the first qubit (creating $(|0\rangle + |1\rangle)/\sqrt{2}$ on the first qubit) followed by a CNOT with the first qubit as control and second as target. This sequence produces the Bell state $\frac{|00\rangle + |11\rangle}{\sqrt{2}}$. The final state vector is approximately $[0.707 + 0j, ; 0, ; 0, ; 0.707 + 0j]$, meaning the system is in an equal superposition of $|00\rangle$ and $|11\rangle$ (with 50% probability each) and zero amplitude for $|01\rangle$ or $|10\rangle$. We visualize the result by plotting the probabilities of each basis state.

Probability distribution of the two-qubit state after creating a Bell state. The bars show the probabilities of each computational basis state $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Only $|00\rangle$ and $|11\rangle$ have nonzero probability (each 0.5), confirming that the final state is $\frac{|00\rangle + |11\rangle}{\sqrt{2}}$.

This is an entangled state – measuring one qubit will immediately determine the other, a hallmark of two-qubit entanglement. The simulator successfully reproduces this state by applying a Hadamard and CNOT in sequence.

Two-qubit circuit: create a Bell state ($|00\rangle + |11\rangle$)/2

Discussion

These examples show that our simulator correctly evolves quantum states under the defined gates. The object-oriented design (with Circuit, Gate, and Qubit classes) makes it easy to build a quantum circuit gate-by-gate and then simulate it. All operations use standard linear algebra via NumPy – for instance, the CNOT gate's action on a 2-qubit state is implemented by grouping amplitudes and applying the 4×4 matrix . The visualization functions provide an intuitive understanding of the results: a single qubit's state is shown on the Bloch sphere (a unit sphere where any pure state corresponds to a point on the surface), while multi-qubit states can be partially understood by their probability distribution over basis states. This simulator can be extended with more gates or used as a foundation for experimenting with small quantum algorithms, all without any specialized quantum computing library. The fidelity of results matches what we expect from theoretical calculations of these quantum circuits, demonstrating the correctness of the implementation.

The definitions of standard quantum gates and their matrices were taken from well-known references . The two-qubit gate matrices